Healthcare Venture Capital Dataset

Overview

This was my first time building a web scraper. Coming from a Java background, working with Python's requests and BeautifulSoup libraries was new to me. Asynchronous programming was also a new concept for me in Python — while I was familiar with concurrency from Java, implementing it with asyncio and aiohttp libraries required learning how Python's event loop works.

This dataset was collected programmatically using Python from two publicly available sources focused on venture capital and startup investors:

- VCSheet
- InvestorHunt

The goal was to gather structured information about healthcare-focused venture capital firms and investors into a single combined dataset.

The final dataset contains approximately 925 unique records after deduplication.

Data Collection Process

Source 1 — VCSheet

The first scraper collected data from VCSheet's healthcare funds directory.

Using the 'requests' library and 'BeautifulSoup', the script:

1. Requested the main healthcare investors listing page
2. Extracted name of each firm and link to individual profile pages
3. Visited each profile page separately
4. Parsed structured fields from each page

The scraper collected fields including:

- Firm name
- Website
- Investment focus
- Stage focus
- Location
- Geographic focus
- Average check size
- Whether the firm leads rounds
- Portfolio examples
- Source link

Each field was stored into a dictionary which then was converted into a csv using the ‘pandas’ library.

Because this source only contained around 43 firms, the scraper was implemented synchronously using standard HTTP requests.

A short delay (time.sleep) was added between requests to avoid overwhelming the site or triggering rate limits.

Source 2 — InvestorHunt

The second scraper collected healthcare investor data from InvestorHunt.

This source contained significantly more records spread across 42 paginated pages, so the scraper was implemented asynchronously using:

- asyncio
- aiohttp

The script:

1. Visited each listing page
2. Extracted investor summary information
3. Collected links to profile page of each investor
4. Requested multiple detail pages concurrently
5. Parsed additional investor information from those pages

This source provided fields such as:

- Investor name

- Location
- Investment focus
- Investor type
- Stage focus
- Portfolio examples
- Source link

Similarly, each field was stored into a dictionary which then was converted into a csv using the ‘pandas’ library.

Using asynchronous requests significantly reduced total runtime compared to a fully sequential approach.

Data Structuring and Merging

Each source was initially exported into its own CSV file.

Because the two sources used different schemas and naming conventions, several normalization steps were required before combining them:

- ‘firm_name’ was renamed to name
- Missing columns were added where necessary
- A ‘data_source’ column was added to preserve traceability
- Placeholder missing values were converted to null values

The datasets were then combined using ‘pandas.concat()’ and deduplicated on the name column.

The final dataset contains fields such as:

- Name
- Website
- Location
- Investment focus
- Stage focus
- Portfolio examples
- Geographies
- Average check size
- Source links

- Data source

Not every source provided every field, so some rows contain null values where information was unavailable.

Challenges Encountered

- **Inconsistent HTML structures across pages:** Different investor profiles stored information in different formats (nested divs, pill components, domain lists, etc.), requiring custom selectors and fallback extraction logic for certain fields.
- **JavaScript-rendered portfolio data:** Some portfolio company names were visible in the browser but not present in the raw HTML returned by requests. Since BeautifulSoup cannot execute JavaScript, some portfolio fields could not be extracted directly without browser automation tools like Selenium or Playwright.
- **Pagination debugging:** InvestorHunt used zero-based page indexing (page=0) rather than the more common one-based indexing, which initially caused empty scrape results until the URL structure was inspected more carefully.
- **Cloudflare bot protection:** A third source (healthcarevclist.com) blocked automated scraping requests with HTTP 403 responses due to Cloudflare protection, preventing it from being used in the final dataset.
- **Schema differences between sources:** The two datasets did not provide identical fields or column structures, so schemas had to be standardized before merging into a unified dataset.
- **Missing or incomplete source data:** Some firms did not provide complete information for fields such as location, portfolio examples, or investment stages. Missing values were preserved as nulls rather than inferred.
- **Performance considerations:** The larger InvestorHunt dataset required asynchronous requests using ‘asyncio’ and ‘aiohttp’ to reduce scrape time and improve efficiency compared to sequential requests.

Potential Improvements

Several improvements could make this pipeline more scalable and production-ready:

- Convert all scrapers to asynchronous workflows:
    - The InvestorHunt scraper already uses ‘`asyncio`’ and ‘`aiohttp`’ to improve performance when processing many pages and detail requests. Applying the same asynchronous architecture to the VCSheet scraper and future data sources would reduce total runtime and improve scalability for larger datasets.
- Adding retry logic and exponential backoff for failed requests
- Using Selenium or Playwright for JavaScript-rendered content
- Storing data in a database such as PostgreSQL instead of CSV files would improve querying, deduplication, incremental updates, and long-term dataset management.
- Scheduling recurring data-collection jobs using tools like Airflow
- Expanding coverage with additional data sources